

Convolutional Neural Networks (CNNs)

Theory, Mathematics, and Architecture Design

Prof. D.Sc. BARSEKH-ONJI Aboud

Facultad de Ingeniería
Universidad Anáhuac México

February 27, 2026

Agenda

- 1 Introduction
- 2 Concepts of Convolutional Neural Networks
 - Basic Concepts
 - Why not Standard MLPs?
- 3 Network Layers
 - Convolutional Layer
 - The Convolution Operation
 - Pooling Layer
 - Activation Function
 - Fully Connected Layer (FC)
- 4 Loss Functions
- 5 Training process of Convolutional Neural Network

Introduction

- 1959: Hubel and Wiesel described that the neurons inside the brain of a cat are organized in layered form.
- 1980: Fukushima proposed the Neocognitron, the first CNN architecture.
- 1989: LeCun et al. proposed the LeNet-5, the first successful CNN architecture.
- 2012: AlexNet won the ImageNet challenge, demonstrating the power of CNNs.

Fukushima's Neocognitron

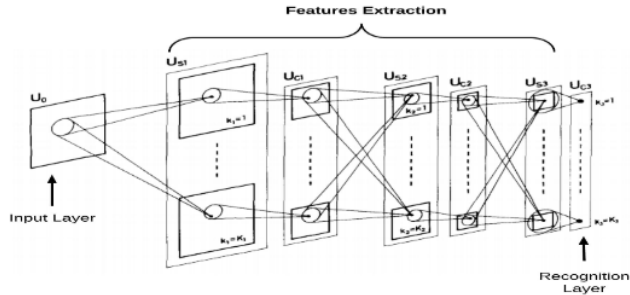


Figure 1: Fukushima's Neocognitron

Agenda

- 1 Introduction
- 2 Concepts of Convolutional Neural Networks
 - Basic Concepts
 - Why not Standard MLPs?
- 3 Network Layers
 - Convolutional Layer
 - The Convolution Operation
 - Pooling Layer
 - Activation Function
 - Fully Connected Layer (FC)
- 4 Loss Functions
- 5 Training process of Convolutional Neural Network

Basic Concepts

Convolutional Neural Network (CNN)

Convolutional Neural Network (CNN), also called ConvNet, is a type of Artificial Neural Network(ANN), which has deep feed-forward architecture and has amazing generalizing ability as compared to other networks with FC layers, it can learn highly abstracted features of objects especially spatial data and can indentify them more efficiently.

Basic Concepts

The CNN Model

A deep CNN model consists of a finite set of processing layers that can learn various features of input data (e.g. image) with multiple level of abstraction.

- The initiatory layers learn and extract the high level features (with lower abstraction),
- and the deeper layers learns and extracts the low level features (with higher abstraction).

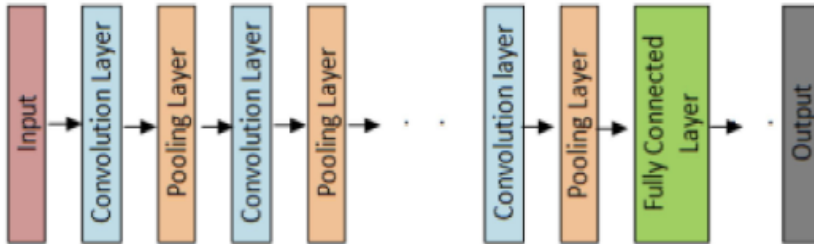


Figure 2: Conceptual mode of CNN

Why not Standard MLPs?

Imagine an image of 224×224 pixels (RGB).

The Curse of Dimensionality

If we flatten this image into a vector:

$$\text{Input Vector} = 224 \times 224 \times 3 = 150,528 \text{ inputs}$$

If the first hidden layer has 1,000 neurons (Dense Layer):

$$\text{Weights} = 150,528 \times 1,000 \approx 150 \text{ Million Parameters!}$$

Problem: Huge computational cost and massive overfitting. **Solution:** Local

CNN vs. Classical NN in the context of computer vision

- One of the main reason for considering CNN in such case is the weight sharing feature of CNN, that reduce the number of trainable parameters in the network, which helped the model to avoid overfitting and as well as to improved generalization.
- In CNN, the classification layer and the feature extraction layers learn together, that makes the output of the model more organized and makes the output more dependent to the extracted features.
- The implementation of a large network is more difficult by using other types of neural networks rather than using Convolutional Neural Networks.

Agenda

- 1 Introduction
- 2 Concepts of Convolutional Neural Networks
 - Basic Concepts
 - Why not Standard MLPs?
- 3 Network Layers
 - Convolutional Layer
 - The Convolution Operation
 - Pooling Layer
 - Activation Function
 - Fully Connected Layer (FC)
- 4 Loss Functions
- 5 Training process of Convolutional Neural Network

Network Layers

As we mentioned earlier, that a CNN is composed of multiple building blocks (known as layers of the architecture), in this subsection, we described some of these building blocks in detail with their roll in the CNN architecture.

- Convolutional Layer
- Pooling Layer
- Activation Layer
- Fully Connected Layer

Convolutional Layer

Convolutional layer1 is the most important component of any CNN architecture. It contains a set of convolutional kernels (also called filters), which gets convolved with the input image (N-dimensional metrics) to generate an output feature map.

Kernel and Convolution Operation

A kernel can be described as a grid of discrete values or numbers, where each value is known as the weight of this kernel. During the starting of training process of an CNN model, all the weights of a kernel are assigned with random numbers (different approaches are also available there for initializing the weights). Then, with each training epoch, the weights are tuned and the kernel learned to extract meaningful features

Kernel and Convolution Operation

The Mathematical Definition

In Deep Learning, a "convolution" is technically a **sliding dot product** (cross-correlation).

We pass a filter (kernel) K over an image I :

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n) \quad (1)$$

- **Element-wise multiplication:** Between the kernel weights and the image patch.
- **Summation:** Aggregating the result into a single pixel in the Feature Map.

Convolution Operation - example

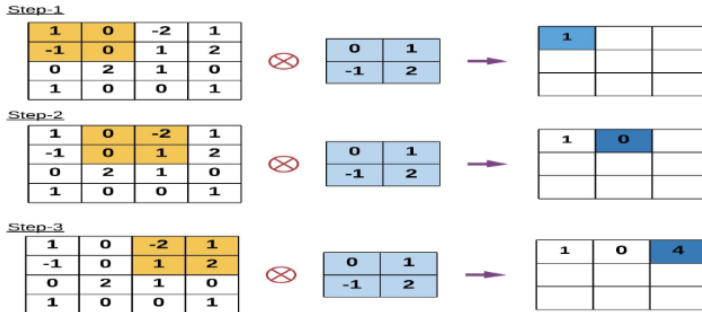


Figure 3: Convolution Operation

Spatial Dimensions Control

How do we calculate the output size of a layer? This is the fundamental arithmetic equation for CNN design.

$$O = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1 \quad (2)$$

Where:

- W : Input Volume Size (Width/Height)
- F : Filter /Kernel Size (e.g., 3×3)
- S : **Stride** (Step size of the slide)
- P : **Padding** (Zeros added around the border)

Padding Options

1. Valid Padding ($P = 0$):

- No padding.
- The image shrinks after convolution.
- Losing edge information.

2. Same Padding ($P > 0$):

- Padding added to preserve dimensions.
- Output Size = Input Size (if stride=1).
- Allows for very deep networks without vanishing spatial size.

MATLAB syntax: "Padding", "same" vs "valid", 0

The Convolution Operation

Padding with Stride - example

Step-1

0	0	0	0	0	0
0	1	0	-2	1	0
0	-1	0	1	2	0
0	0	2	1	0	0
0	1	0	0	1	0
0	0	0	0	0	0



1	0	1
0	-1	2
-2	1	0



-2	

Step-2 :

0	0	0	0	0	0
0	1	0	-2	1	0
0	-1	0	1	2	0
0	0	2	1	0	0
0	1	0	0	1	0
0	0	0	0	0	0



1	0	1
0	-1	2
-2	1	0



-2	-1

Figure 4: Padding with Stride

Main advantages of Padding

The padding is important to give border size information of the input image more importance, otherwise without using any padding the border side features are gets washed away too quickly. The padding is also used to increase the input image size, as a result the output feature map size also get increased.

Pooling Layer

Pooling layer is used to reduce the spatial dimensions of the feature maps. It is also used to reduce the number of parameters in the network, which helps to avoid overfitting and as well as to improved generalization.

Note

The pooling operation is performed by specifying the pooled region size and the stride of the operation, similar to convolution operation.

Pooling Layer

Different types of Pooling

here are different types of pooling techniques are used in different pooling layers such as max pooling, min pooling, average pooling, gated pooling, tree pooling, etc. Max Pooling is the most popular.

Pooling Operation - example



Figure 5: Pooling Operation

Size after Pooling

The formula for calculating the output size of a pooling layer is:

$$h' = \left\lfloor \frac{h - f}{s} \right\rfloor \quad (3)$$

$$w' = \left\lfloor \frac{w - f}{s} \right\rfloor \quad (4)$$

Where:

- h : Input Feature Map Height
- w : Input Feature Map Width
- f : Pooling region size (e.g., 3×3)
- s : **Stride** (Step size of the slide)

Activation Function

Activation function is used to introduce non-linearity in the network, which helps to learn complex patterns in the data.

In CNN architecture, after each learnable layers (layers with weights, i.e. convolutional and FC layers) non-linear activation layers are used. The non-linearity behavior of those layers enables the CNN model to learn more complex things and manage to map the inputs to outputs non-linearly. The important feature of an activation function is that it should be differentiable in order to enable error backpropagation to train the model. The most commonly used activation functions in deep neural networks (including CNN) are described below.

Fully Connected Layer

The Fully-Connected Layers are type of feed-forward artificial neural network (ANN) and it follows the principle of traditional multi-layer perceptron neural network (MLP). The FC layers take input from the final convolutional or pooling layer, which is in the form of a set of metrics (feature maps) and those metrics are flattened to create a vector and this vector is then fed into the FC layer to generate the final output of CNN.

Flattening & Fully Connected

Transition to Classification

After extracting spatial features with Convolution+Pooling, we get a 3D volume (e.g., $7 \times 7 \times 512$).

- 1 **Flatten:** Unroll the volume into a 1D vector (size 25,088).
- 2 **Fully Connected (FC):** Standard MLP layers to combine features and perform classification logic.
- 3 **Softmax:** Probability distribution for the output classes.

Agenda

- 1 Introduction
- 2 Concepts of Convolutional Neural Networks
 - Basic Concepts
 - Why not Standard MLPs?
- 3 Network Layers
 - Convolutional Layer
 - The Convolution Operation
 - Pooling Layer
 - Activation Function
 - Fully Connected Layer (FC)
- 4 Loss Functions
- 5 Training process of Convolutional Neural Network

Loss Functions

Loss Functions

Loss functions are used to measure the difference between the predicted output and the actual output.

- **Cross-Entropy Loss or Softmax Loss:** Used for classification tasks.
- **Euclidean Loss Function:** Used for regression tasks.

Agenda

- 1 Introduction
- 2 Concepts of Convolutional Neural Networks
 - Basic Concepts
 - Why not Standard MLPs?
- 3 Network Layers
 - Convolutional Layer
 - The Convolution Operation
 - Pooling Layer
 - Activation Function
 - Fully Connected Layer (FC)
- 4 Loss Functions
- 5 Training process of Convolutional Neural Network

Training process of Convolutional Neural Network

The training process mainly includes the following steps:

- 1 Data pre-processing and Data augmentation.
- 2 Parameters initialization.
- 3 Regularization of CNN.
- 4 Optimizer selection.

Parameters initialization

The most easiest way to doing it is by initializing all the weights with zero. However, This turns out to be a mistake, because if we initialize weights of all layer to zero, the output as well as the gradients (during backpropagation) calculated by every neuron in the network will be the same. Hence the update to all the weights would also be the same.

Parameters initialization

Different Initialization Techniques

- Random Initialization
- Xavier Initialization
- Unsupervised Pre-training

Agenda

- 1 Introduction
- 2 Concepts of Convolutional Neural Networks
 - Basic Concepts
 - Why not Standard MLPs?
- 3 Network Layers
 - Convolutional Layer
 - The Convolution Operation
 - Pooling Layer
 - Activation Function
 - Fully Connected Layer (FC)
- 4 Loss Functions
- 5 Training process of Convolutional Neural Network

Recent advancement in CNN architectures

LeNet-5 Key note:

- The LeNet-5 was trained on the MNIST digit dataset.
- It used sigmoid non-linearity as the activation function.
- It used stochastic gradient descent (SGD) learning algorithm with 20 training epoch.
- It used 0.02 as the momentum factor value.
- It reduced test error rate to 0.95% on MNIST data set.

LeNet-5 Architecture

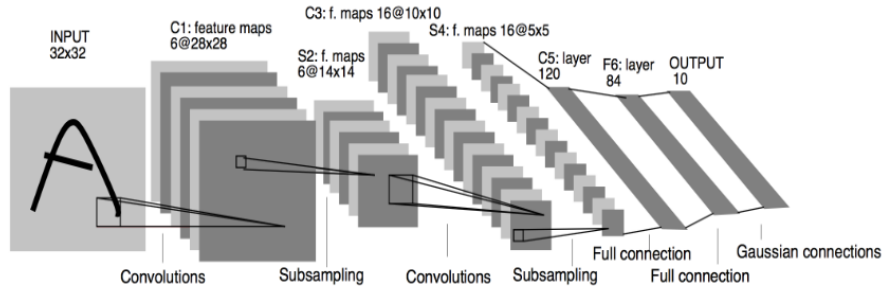


Figure 6: LeNet-5 Architecture